

Table of Contents

- **Introduction (pg. 1)**
- **Executive Summary (pg. 2)**
- **Scenarios (pg. 3 - 4)**
- **Interface Transitions (pg. 3)**
- **Screen by Screen Breakdown (pg.)**
- **Glossary (pg. 4 - End)**

Introduction:

- **Summary of the problem our project solves:**

In robotics, there are many approaches to controlling a robot. Our clients, specifically college professors involved in robotics and mathematics, are looking for a control medium that allows them to maneuver the robot and observe sensor readings.

- **What our project is and how it solves the aforementioned problem:**

The solution our team is implementing is a VR headset controller that can be used to control robot movement and observe sensor output. The main robot we'll be focusing on for this project is an OSOYOO Robot Pie Car equipped with a camera, a voltage meter, a tracking sensor, and an ultrasonic sensor. The headset we'll be using is the Meta Quest 3 equipped with 2 handheld joystick controllers and the actual headset. Our implementation will allow the user to control the robot via the joysticks and read from the camera and sensors using the headset.

Executive Summary:

The Virtual Robotics project is a WebXR Meta Quest application that will mainly aid Professor Hall's and Professor Benedict's research on real-world robotics applications. The main application functions are a live-feed robot camera and controller, as well as a menu to switch the inputs and outputs between multiple different robots to the headset (if a second robot is completed in time, which is outside the scope of this class' work).

The universality of the Pi robots will be a streamlined process that will only require small file modifications to be able to use a new robot that runs on a Raspberry Pi. After the completion of the project, all code and supplies used will be made public, allowing anybody to use, replicate, or expand upon our application.

The live camera footage of the robot camera, as well as the controls, will be used by Dr.Hall and Dr. Benedict to more efficiently control robots that are used for conducting teaching and research. It will consist of a start screen where you select any number of robots you will want to use from a menu selection screen highlighting them, and then a start button. Clicking the start button will then load the visuals of the first robot chosen, where you are shown the camera feed of the robot and a diagnostics see-through overlay that is on the side. This will have important information about the robot from the available sensors. You will also see a drop-down menu in which you will see an end session button to terminate the program along with settings to toggle the overlay on and off.

Scenarios:

- Platform Testing for Different Types of Robots:

Users can adapt the platform to their robot of choice and have a means of testing different movement systems of that robot with a platform that allows them to display any potential sensor readings.

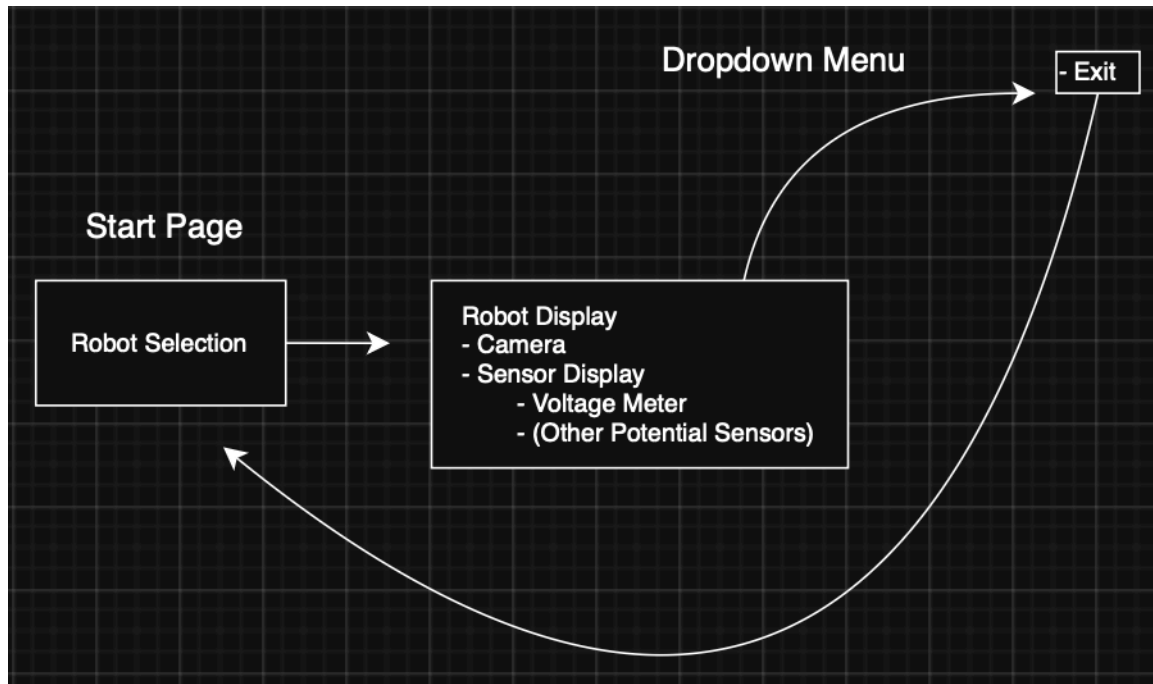
- Environment Exploration

Users of the platform can access a camera feed of whatever sensor on their robot of choice they'd like, and depending on how small their robot of choice is, they can even use the platform to gain access to certain areas or crevices that they'd have a hard time getting into by themselves.

- Recreation

The platform can also be used for entertainment/noneducational purposes, such as having a sensor's camera feed be used for video footage that can be uploaded onto the internet as either part of a funny YouTube video or as makeshift security footage that can be used to spy on insects or small animals(e.g. fruit flies, ants, small dogs/cats, etc.).

Interface Transitions



Screen by Screen Breakdown:

- In the example, this section serves as an explanation of each page the platform offers, going through each button/feature seen on each page. Our screens are not yet developed at this time and whether or not we have screens like you would see on a traditional application is dependent on whether they'll be needed.

Glossary:

Microcontroller:

A microcontroller is a very small computer on an integrated circuit that can decide what processes happen and when on a variety of electronic devices, such as microwaves, thermostats and washing machines. Since it is very small, it's not as powerful as most of the bigger computers are as of this point in time due to having a small amount of memory and limited

software capabilities, but on the brighter side of things, it doesn't use as much electricity as a bigger computer would, either.

Raspberry Pi:

A more powerful genre of microcontrollers (depending on what version you buy) that can do almost everything a regular computer can do while also not costing as much money to buy as that said computer. In this project, the Raspberry Pi will function as a translator between the VR headset and the motors.

Single Board Computer:

Essentially a miniature computer, similar to the ones people use almost every day, except that it is much less powerful (e.g. smaller amounts of memory, more limited software capabilities, etc) with the added benefit of not requiring as much electricity to use.

VR:

An acronym for Virtual Reality, which refers to the concept of an electronic device (typically a headset) using components such as cameras and sensors to run a program that overrides the real world camera feed and replaces it with a fully digitalized scene. In this project, this term will be used to refer to virtual reality headsets such as the Meta Quest and Apple Vision Pro.

AR:

Similar to Virtual Reality in concept, but this time, Augmented Reality(AR) only focuses on making digital objects appear as though they are actually in the real world via the aforementioned cameras and sensors.

Apple Vision Pro:

Apple's proprietary virtual reality platform(which may also double as a Mixed Reality (Virtual Reality(VR) + Augmented Reality(AR)) platform when used for activities outside the scope of this project. Compared to most other virtual reality headsets, all you need to do to control what the Vision Pro does is move your eyes and/or hands, or speak into the headset's microphone with your voice.

Meta Quest:

Meta's proprietary virtual reality platform that no longer requires a PC to run as of the Meta Quest 2's release. It was called the Oculus Quest about a decade ago, but had its name changed when Facebook's parent company changed its name to Meta in 2022 after buying the Oculus VR company on March 25, 2014. Compared to most other virtual reality headsets, you can download the Meta Horizon app on your smartphone to set it up instead of having to rely on a PC.

Router:

Manages data traffic between devices on a network. Will be used in this project to create an isolated network to minimize connection issues between the headset and the device.

IMU (Inertial Measurement Unit):

A small sensor chip that measures how a device is moving and tilting. It usually combines an accelerometer (measures changes in speed) and a gyroscope (measures rotation). Some also include a magnetometer, which works like a compass. In this project, it would report the robot's orientation.

WebXR Gamepad API:

A set of tools built into web browsers that lets a webpage read VR controller inputs, such as joystick position, button presses, and hand pose. This is how the app running in the Quest browser reads the controllers.

Hand Pose:

The position and orientation of the user's hand or controller in 3D space, as tracked by the headset.

JavaScript:

The programming language that runs inside web browsers. Here, it runs in the Quest browser to read the controllers and send commands.

Python Server:

A program written in Python that runs on the Raspberry Pi. It listens for incoming commands from the headset and translates them into motor instructions.

WebSocket:

A type of network connection that stays open, so data can flow in both directions continuously. It

is well suited to real-time control because each joystick update doesn't need a brand-new connection.

WiFi (2.4GHz / 5GHz):

Wireless networking. The 5GHz band is faster and less crowded than 2.4GHz, which helps with low-delay video, but it has a shorter range and is worse at passing through walls.

Network Stack:

The layers of software in a device that handle sending and receiving data over a network.

Network Congestion:

Slowdowns that happen when too many devices or too much data are competing on the same network.

Isolated Network:

A private network, created here by the dedicated router, with only the project's devices on it. It avoids interference from other traffic.

Latency:

The delay between an action and its result, such as the time between moving the joystick and the robot responding, or between the camera capturing a frame and it appearing in the headset. Low latency is critical for comfortable VR control.

CSI Camera:

A camera that plugs directly into the Raspberry Pi's dedicated camera port through the Camera Serial Interface (CSI). It is faster and more efficient than a USB webcam because it connects straight to the Pi's image-processing hardware.

Ribbon Cable:

A flat, wide cable made of many thin wires side by side. It connects the CSI camera to the Raspberry Pi.

PWM (Pulse Width Modulation):

A way of controlling power by switching a signal on and off very quickly. The share of time the signal stays "on" (the pulse width) sets how much power gets through. A wider pulse makes a motor spin faster, and a narrower pulse makes it spin slower.

PWM HAT (PCA9685):

An add-on board that sits on top of the Raspberry Pi. "HAT" stands for Hardware Attached on Top. The PCA9685 chip generates steady, precise PWM signals on many channels at once, so the Pi doesn't have to produce the timing itself.

H-Bridge:

A circuit that can reverse the direction of the current flowing through a motor, which lets the motor spin both forward and backward. Motor driver boards are built around one or more H-bridges.

Motor Driver Board (Model X):

A board that takes the low-power control signals from the PWM HAT and switches the much larger battery current the motors need. The Pi can't power motors directly, so the driver board acts as the middleman.

Gear Motor:

An electric motor with a built-in gearbox. The gears trade speed for torque, which gives the motor enough turning force to move the robot's wheels.

I2C Bus:

A simple two-wire communication system that lets the Raspberry Pi talk to multiple chips, such as the PWM HAT or an IMU. Each device on the bus has its own address, so they can share the same wires.

ISP (Image Signal Processor):

A part of the Pi's chip that turns the raw data from the camera sensor into a usable image. It handles tasks like adjusting color, brightness, and noise.

Ethernet Cable:

A wired network cable. It is useful for setting up the Pi or router when a wireless connection isn't working yet.

H.264 Encoder:

Hardware or software that compresses video using the H.264 standard. It makes the video small enough to send over WiFi quickly. The Pi 4 has a dedicated chip that does this.

Decode:

Decompressing the encoded video on the receiving end so it can be displayed. Here, the Quest browser does the decoding.

Camera Feed:

The live video stream from the robot's camera.

Throttle:

The control that sets forward and backward speed. Here, it is the left joystick.

Yaw:

Rotation left or right around the vertical axis, meaning turning in place or steering. Here, it is controlled by the right joystick.

Orientation:

Which way the robot is facing and how it is tilted. It is usually described as yaw (turning), pitch (nose up or down), and roll (tilting side to side).

Project Management**GitHub Repository:**

An online folder where the project's code is stored, versioned, and shared among team members.

Sprint:

A fixed block of time, often one to two weeks, focused on completing a specific set of goals for a project.